

Lecture 7:

Call Stack

- 7.1 Background
- 7.2 Stack (aka Execution Stack, Call Stack)
- 7.3 Compromising control flow

Summary and takeaways

- Demonstrate how process integrity can be compromised by modifying values in memory.
- Call stack facilitates function calls. It keeps track of different instances of function call. Without that, we won't have modular software design.
- An object in the stack is a “stack frame” and each frame contains info of the environment an instance of the function operates. Info include control flow info (return address), local variables, and parameters. Malicious modification of those info would have significant consequence (next lecture on stack smash).

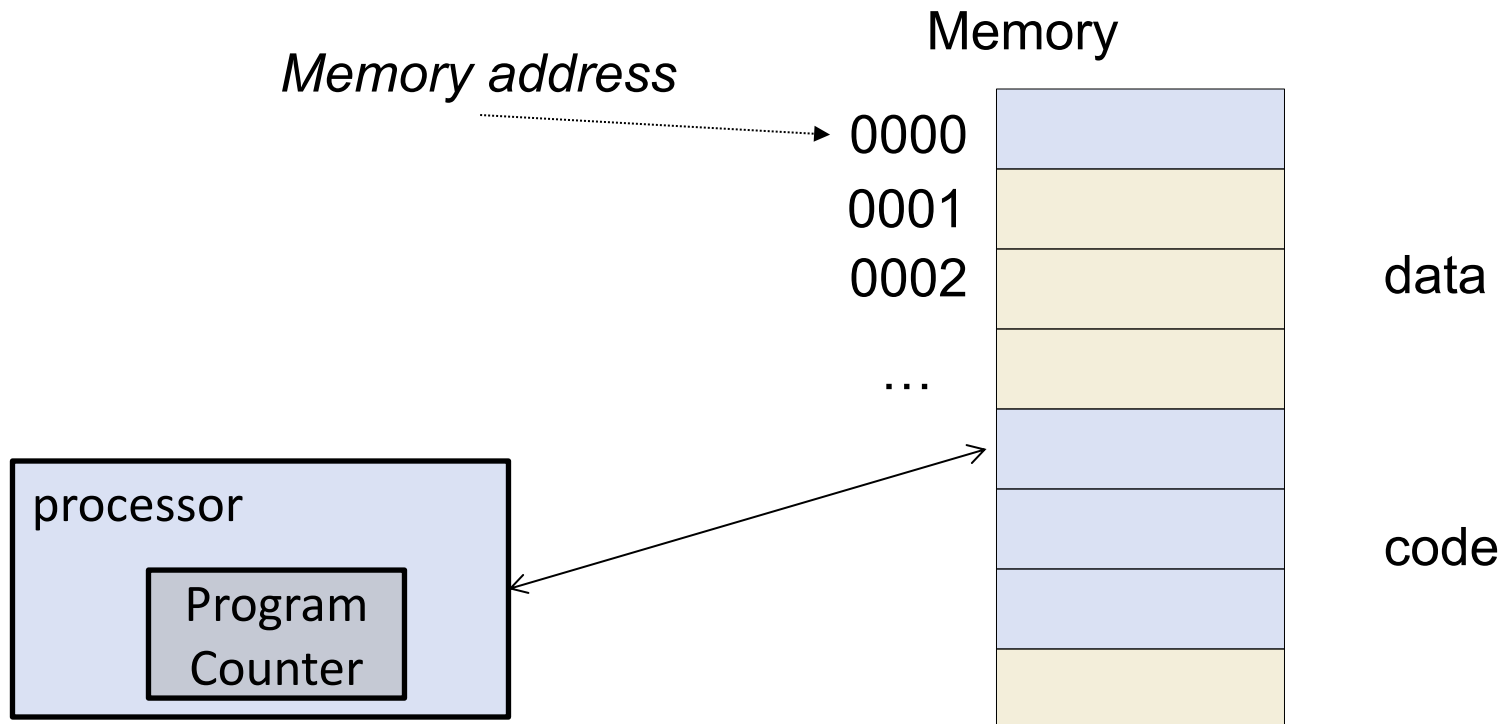
7.1 background

Many concepts in software security requires good understanding on how a piece of code is executed. This lecture covers callstack.

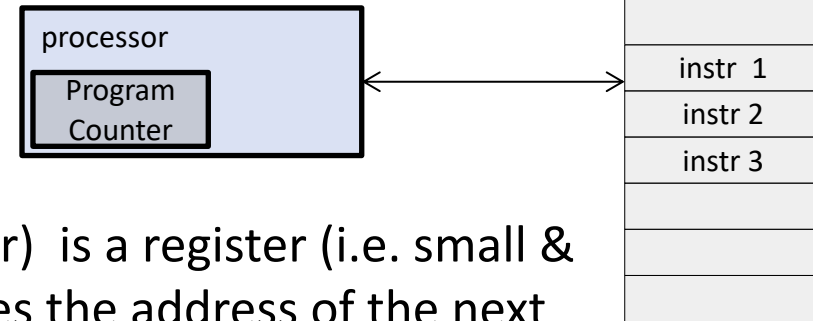
Not covered in this lecture include memory management, design of OS, etc.

Code vs Data

- Modern computers are based on the *Von Neumann computer architecture*. The *code* are treated as *data* and both are stored in the same memory. There are no clear distinction of code and data. The “program counter” indicate location of next instruction to be executed. (In contrast, the *Harvard architecture* has separated hardware component that stores the code and data).



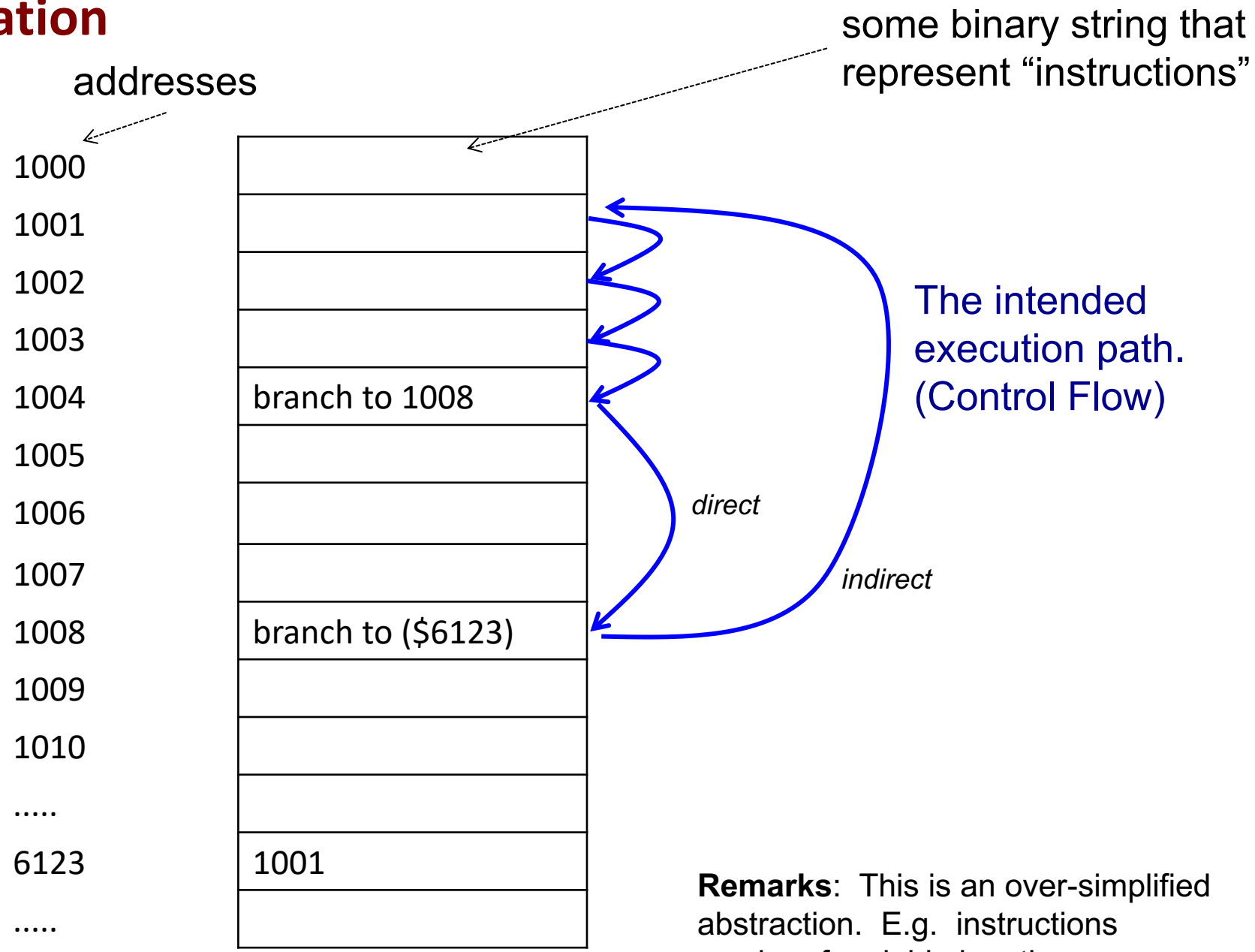
Control Flow



- The program counter (aka Instruction Pointer) is a register (i.e. small & fast memory within the processor) that stores the address of the next instruction.
- After an instruction is executed, the processor fetches another instruction from the memory, whose address is specified in the program counter. After the new instruction is fetched, the program counter automatically increases by 1, and then execute the fetched instruction. In this way, instructions would be executed sequentially in the memory.
- During execution, program counter could also be changed by the fetched instruction, e.g*,
 1. (direct branch) replaced by a constant value specified in the instruction;
 2. (indirect branch) replaced by a value fetched from the memory. There are many different forms of indirect branch.

*: for simplicity, we omit conditional branch and call/return here.

illustration



Remarks: This is an over-simplified abstraction. E.g. instructions can be of variable length.

7.2 Stack (aka Execution Stack, Call Stack)

See: https://en.wikipedia.org/wiki/Call_stack

Functions

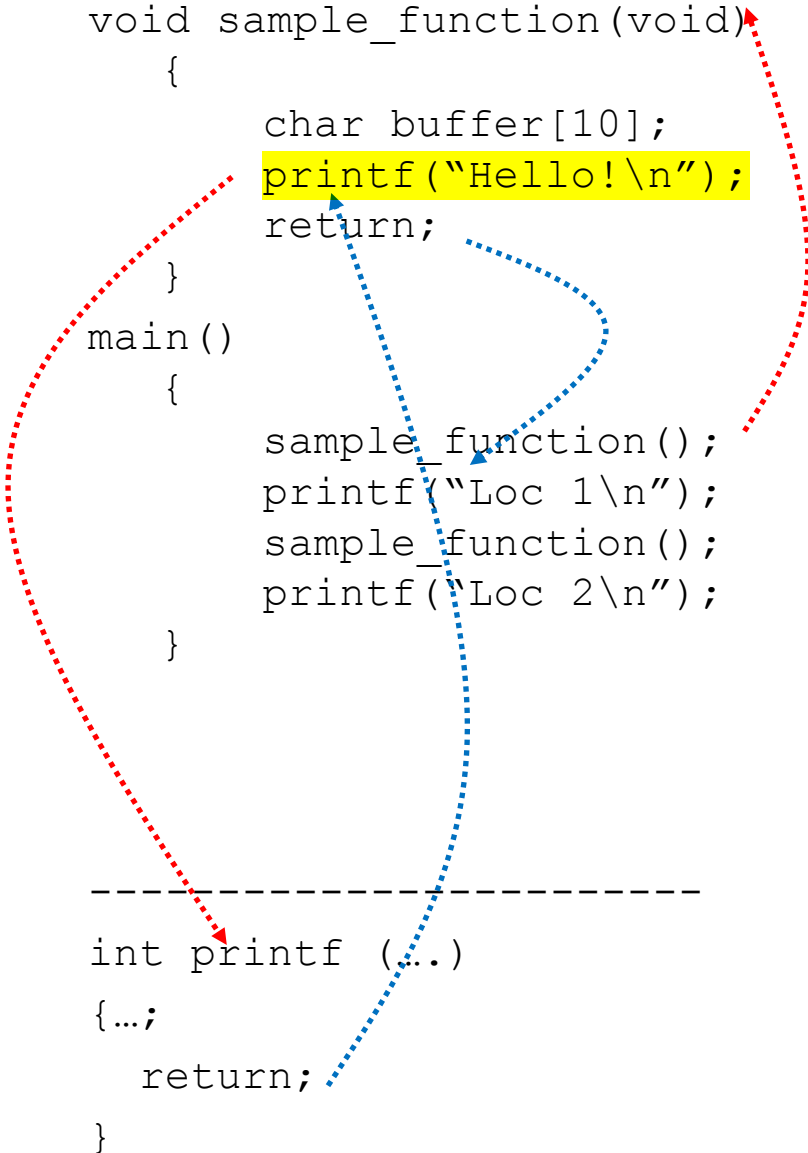
- Subroutine/function/procedure breaks code into smaller pieces:
 - Facilitate modular design and code reuse
- A function can be called from different part of the program, and even recursively.
- **Question 1:** how does the system knows where it should return to after a function is completed?
- **Question 2:** where are the function's *arguments* and *local variables* stored?
- These are managed by “call stack”.

```
void sample_function(void)
{
    char buffer[10];
    printf("Hello!\n");
    return;
```

```
main()
{
```

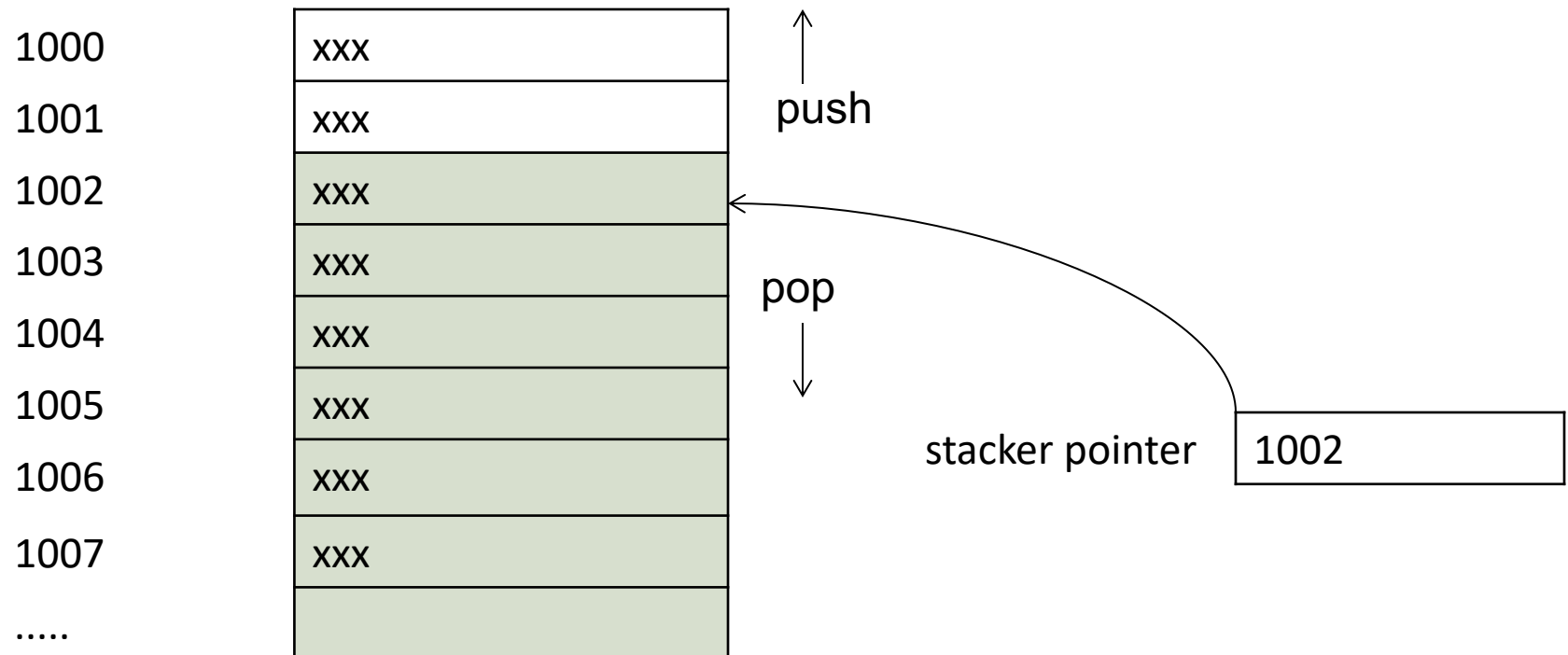
```
    sample_function();
    printf("Loc 1\n");
    sample_function();
    printf("Loc 2\n");
}
```

```
int printf (...)
{...;
    return;
}
```



Remark: Stack

- Stack is a data structure. It is not a separate hardware.
- A call-stack is a stack that store important information of the function calls. Each element of the stack is called a stack frame.
- *Stack pointer* is a variable that stores location* of the first element. There are two operations in stack: Push and Pop. (Last-In-First-Out).



*: A value that is the location of memory is often referred as a “pointer”. It “points” to a memory.

Note: In this example, the stack grows “upward”.

Call stack

- Recap that Stack is last-in-first-out. This makes it useful in maintaining function calls.
- During execution, a stack is maintained to keep control flow information and passing of parameters. This stack is known as **Call stack**.
- For an instance of a function call, the call stack keep tracks of
 - Parameters to be passed
 - Control flow information: return addresses (i.e. who invoke this function)
 - Local variables of functions
- Each call/invocation of a function pushes an activation record (aka **stack frame**) to the stack, which contains:
 - parameters,
 - return address,
 - local variables, and
 - pointer to the previous stack frame in the stack. (this is included for efficiency. Conceptually, this piece of info is not required to maintain the control flow. Let's ignore the role of this. We mentioned it here since most documents mentioned this)

Illustration.

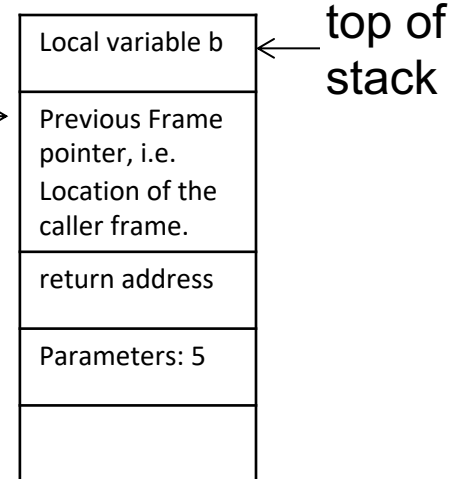
When a function is called, the parameters, return address, local variables are “pushed” into the stack.

E.g. for the following segment of C program:

```
int Drawline( int a)
{int b =1;
..
}

int main()
{
    Drawline (5);
}
```

Frame pointer
(due to some efficiency consideration, **frame pointer** points here instead of top-of-stack. Call be ignored for simplicity. Mentioned here so that you won't get confused when reading other documents.)



When the function `Drawline(5)` is invoked, the followings steps are carries out:

- (1) These data are pushed into the stack in this order:
 - the parameter (which is 5),
 - the return address (i.e. value of program counter), and
 - the value of the local variable b (which is 1).
- (2) The control flow branches to the code of “Drawline”.
- (3) Execute “Drawline”.
- (4) After “Drawline” is completed, pops out the variable, return address and parameter.
- (5) Control flow branches to “return address”.

```

int func1( int a)
{int b =253;
    func2( 2);
}

int func2( int a)
{int b =15;
    func3( 1);
}

int func3( int a)
{int b =0;

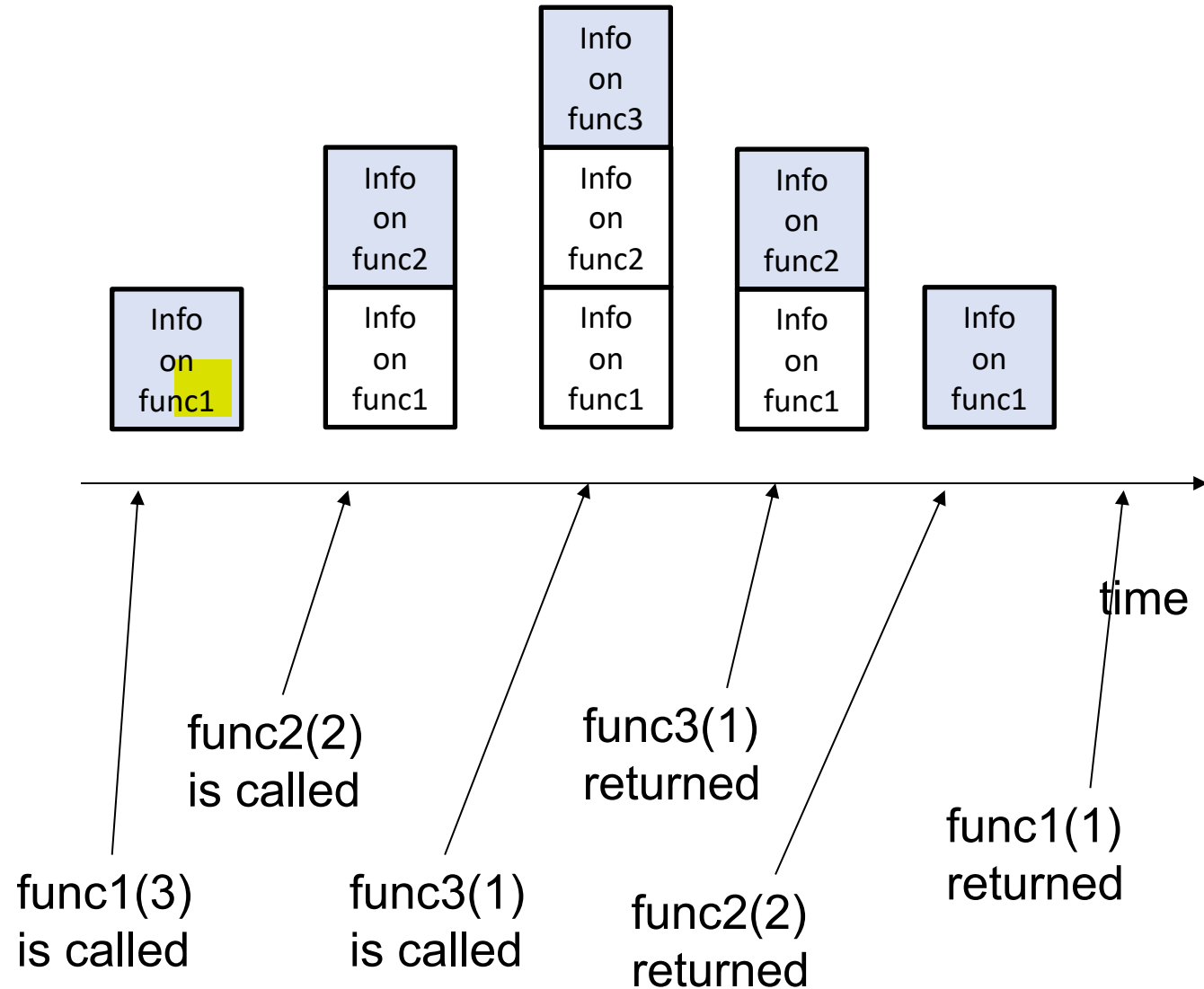
```

```

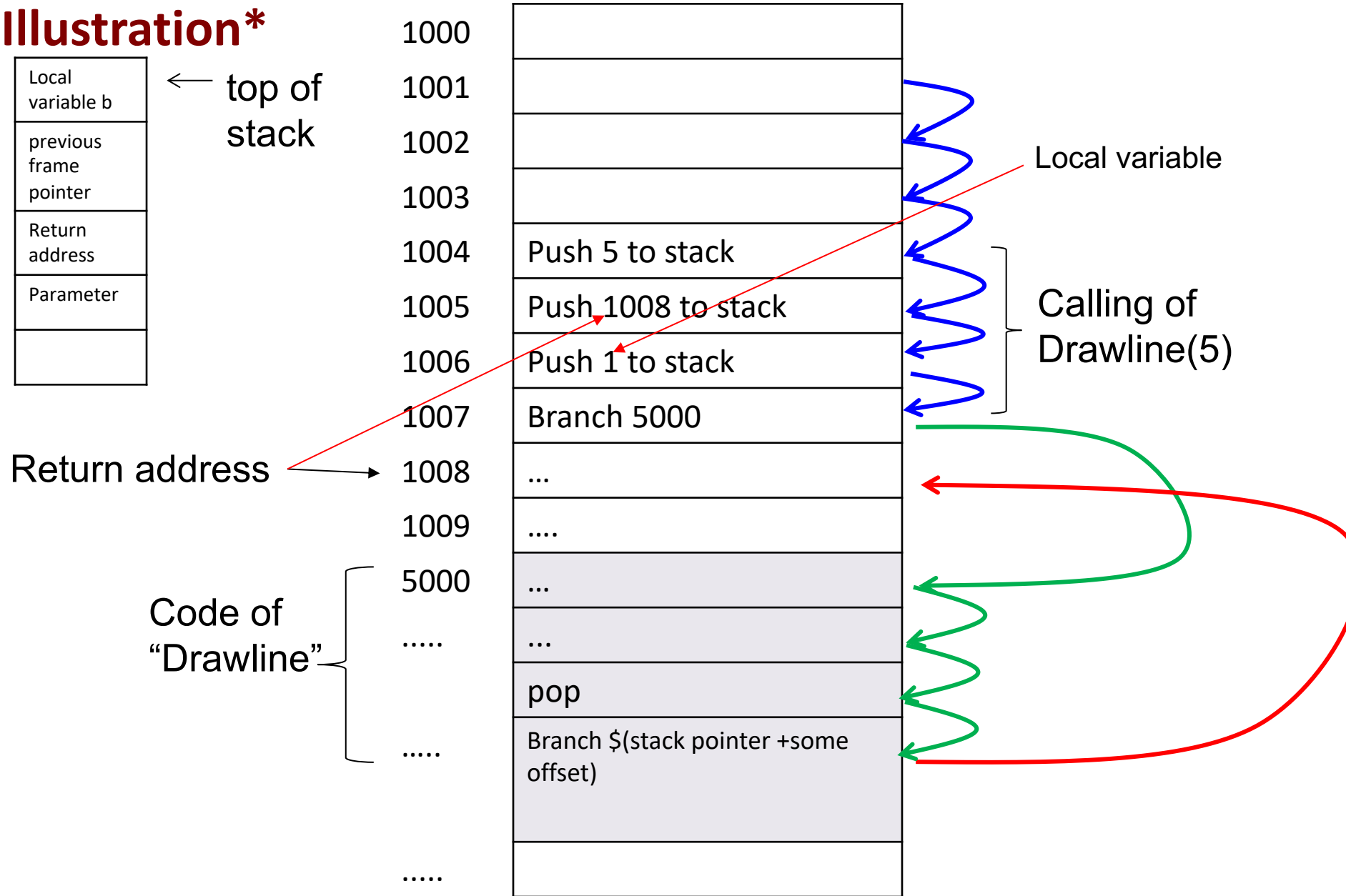
}

int main()
{
    func1(3);
}

```



Illustration*



*: This slide gives a simplified view. Actual implementation includes "return value" and involves the "frame pointer".
For more details, see <http://www.tenouk.com/Bufferoverflowc/Bufferoverflow2a.html> or https://en.wikipedia.org/wiki/Stack_buffer_overflow

8.2 Control flow integrity

Remarks: implications on security.

- We have seen how the call stack stores the the *return address* as data in the memory.
- Code is stored as data.
- Attacker could compromise the execution integrity by either modifying the code or modifying the control flow, in particular, the return address in the call stack.

Compromising memory integrity → control flow integrity

- Let's assume that the attacker has the capability to write to some memory (i.e. able to compromise ***memory integrity***). In the next few slide, we show that this can lead to compromise of the ***control flow integrity***.
- E.g., by exploiting buffer overflow (if the program is not carefully written and allow that), the attacker could write to certain memory that the attacker originally doesn't have access to.
- While possible, it is still not so easy for an attacker to compromise memory. In addition, it may come with some restrictions. For e.g. attackers can only write to some particular location, or the attacker can only write a sequence of consecutive bytes, or the attacker can write but not read, etc. In other words, although we assume that attacker can compromise memory, keep in mind that the attackers unlikely to have arbitrary access to any memory.

Possible Attack Mechanisms

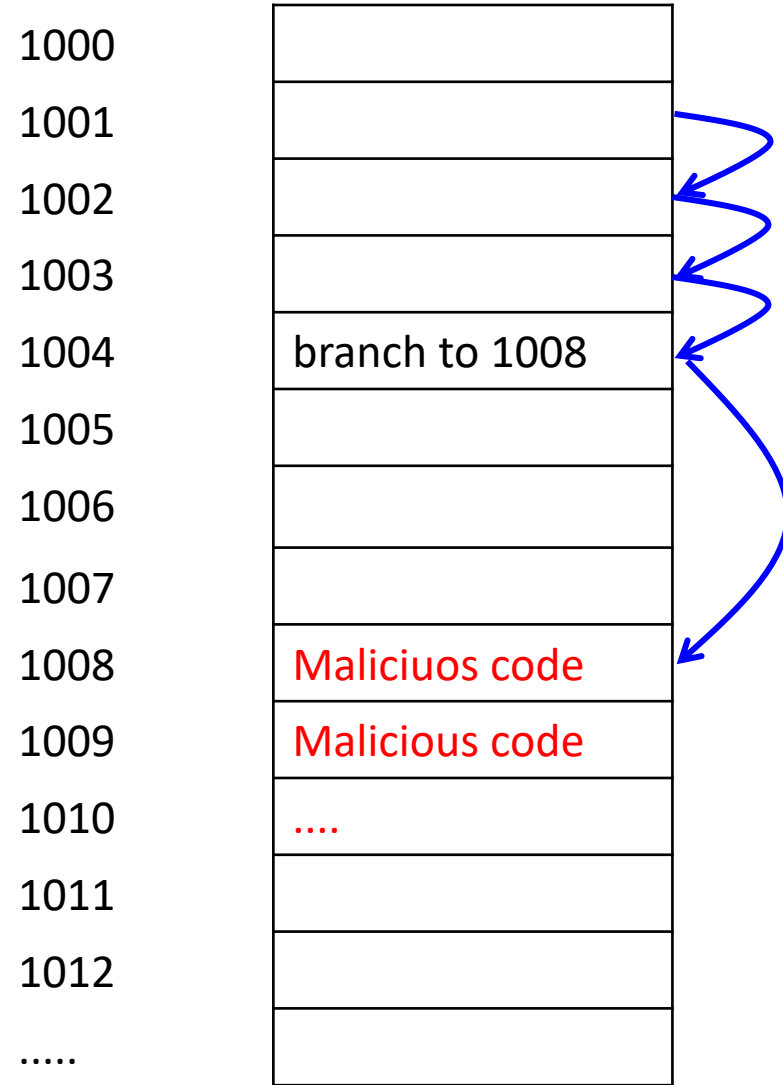
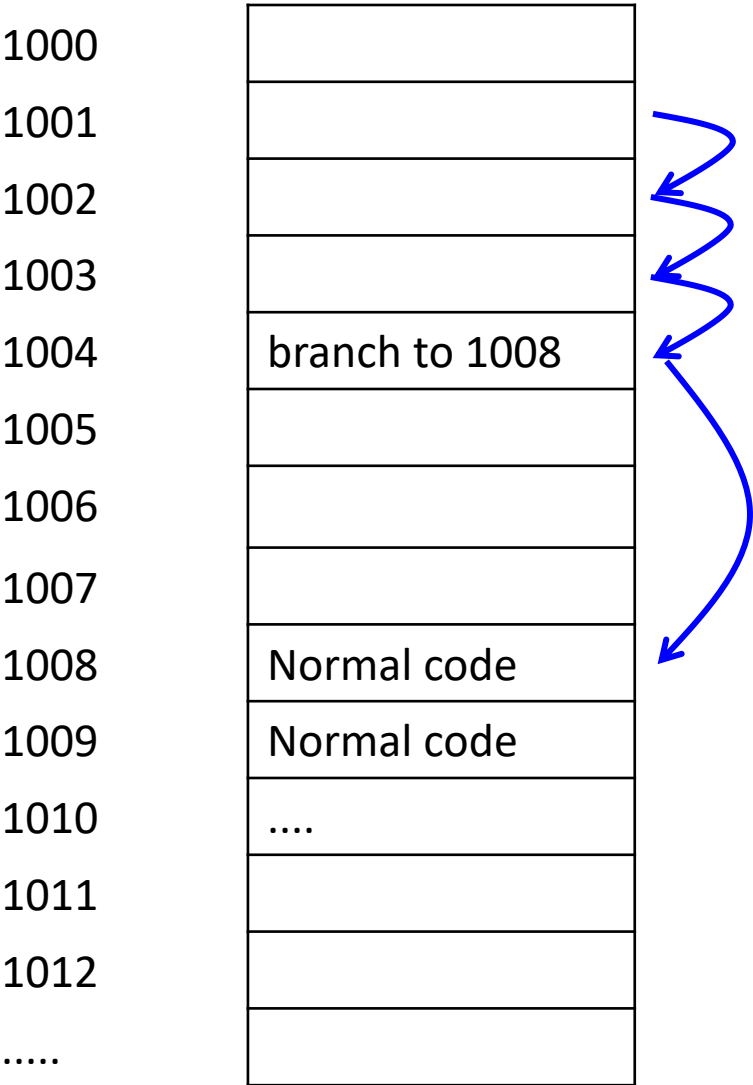
Let us assume that the attacker has the capability to *write* to some memory locations and wants to compromise the execution integrity. The attacker could:

- (Attack 1) Overwrite ***existing execution code portion*** with malicious code; or
- (Attack 2) Overwrite a piece of control-flow information:
 - (2a) Replace a ***memory location*** storing a code address that is used by a *direct jump*
 - (2b) Replace a ***memory location*** storing a code address that is used by an *indirect jump*

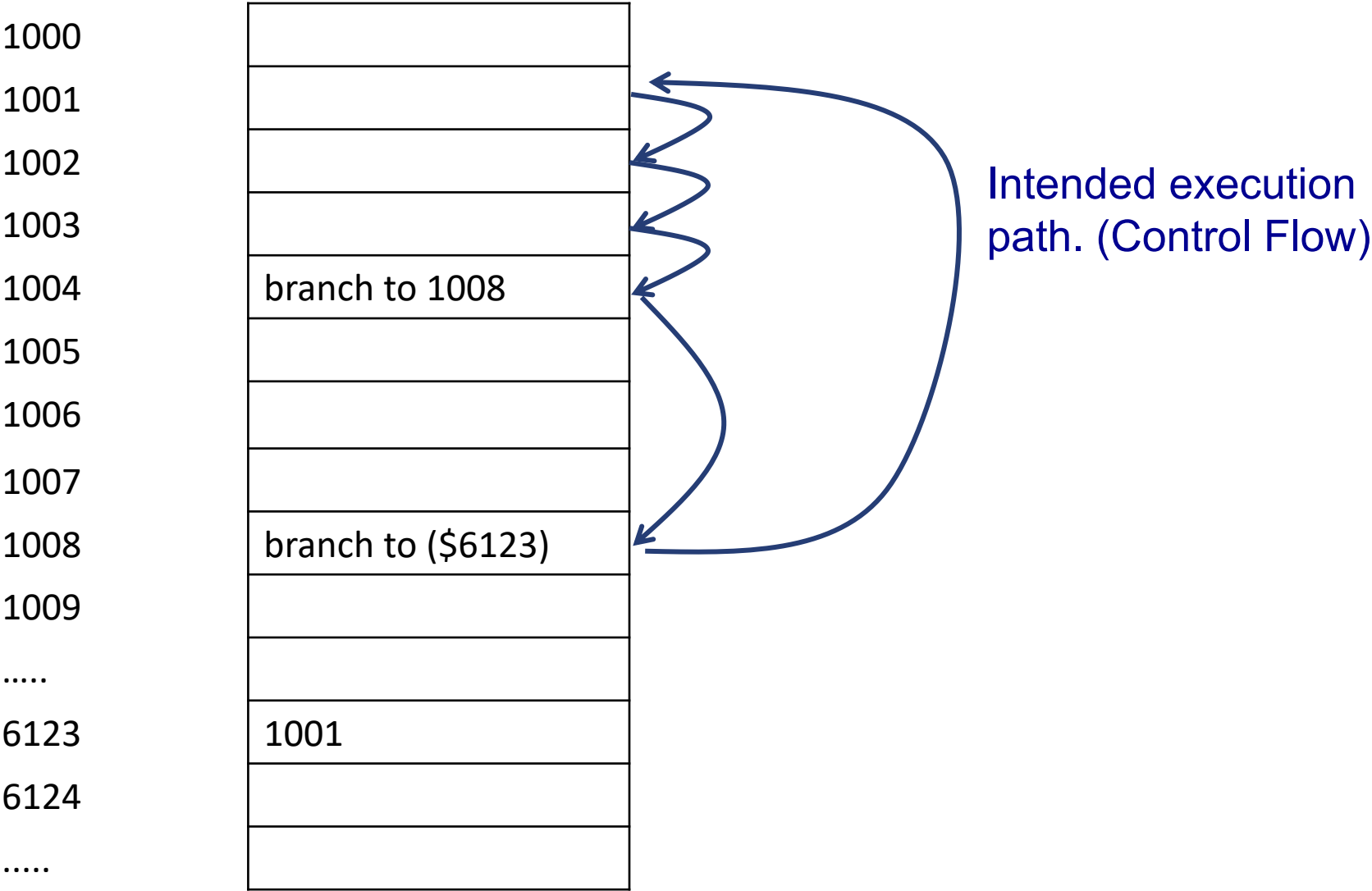
The above three attacks are illustrated in the next few slides.

When Attack (2b) can be carried out on stack, it is aka ***Stack smashing***. (next lecture on buffer overflow).

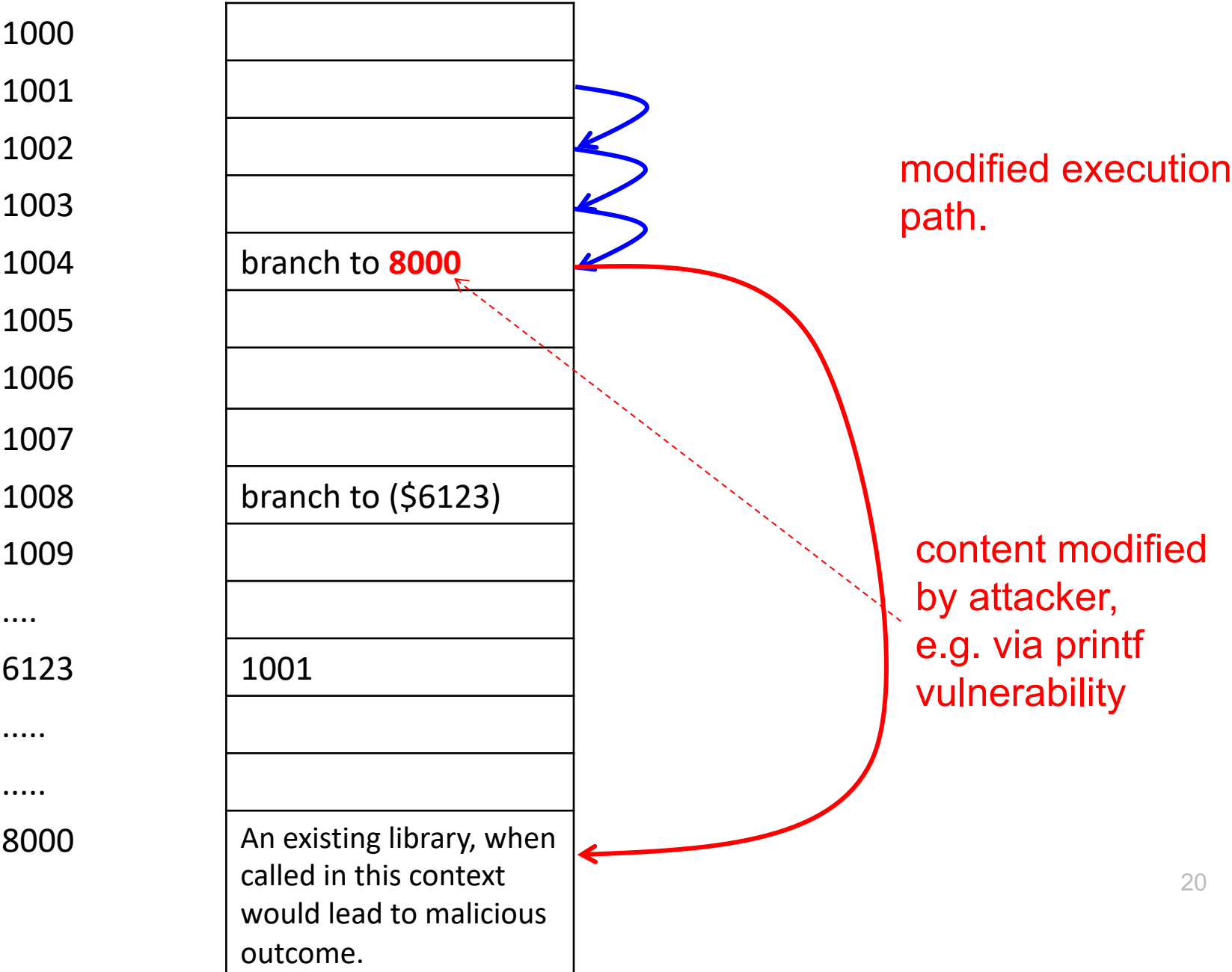
Attack 1 (replace the code)



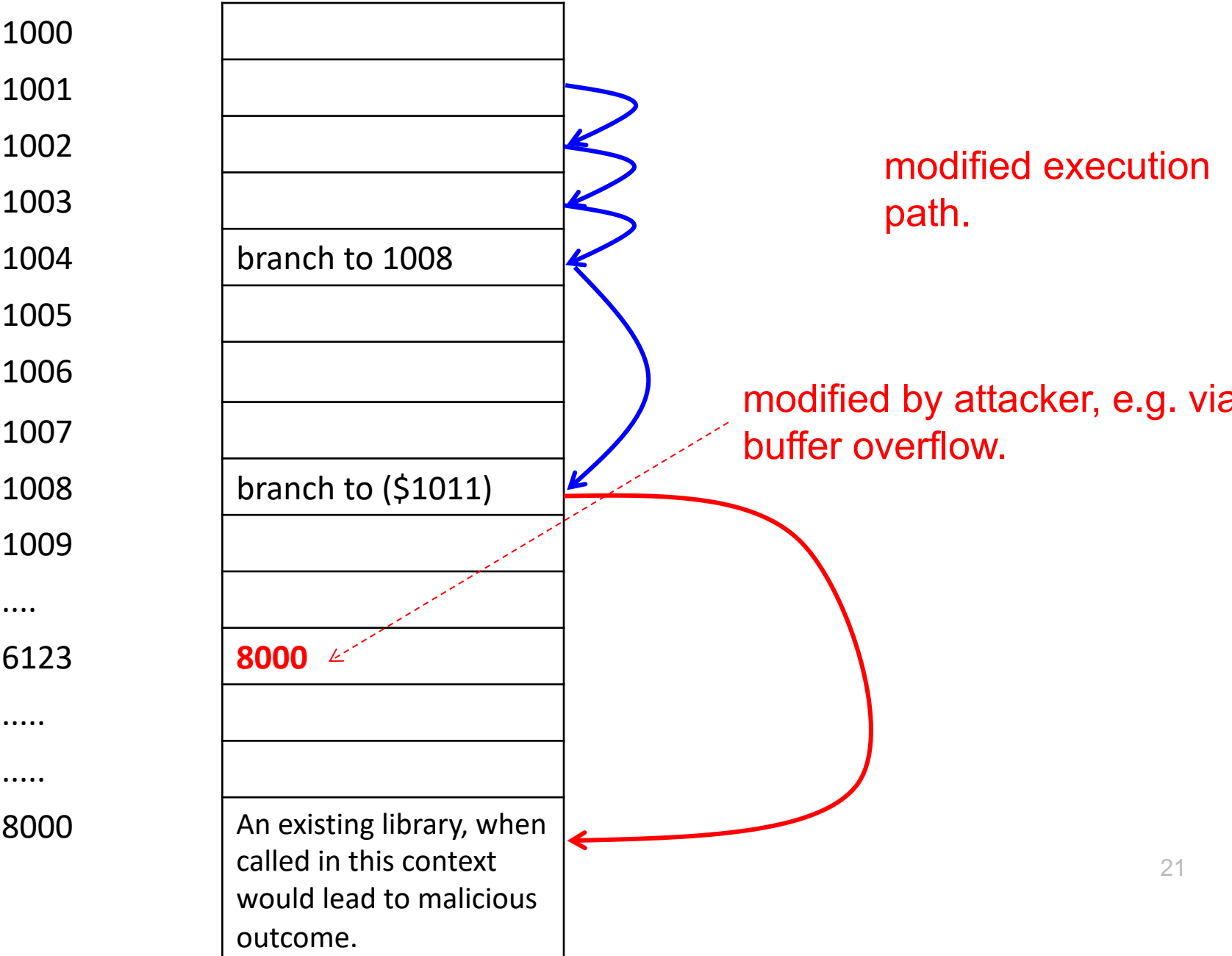
Attack 2a, 2b: Normal control flow before being attacked.



Attack 2a (Memory locations that store the code being modified)



Attack 2b (Memory locations that store the addresses being modified)



Remark: Process Memory Layout

Optional

- Simplified Linux process memory showing various segments:
- (Optional:
http://dirac.org/linux/gdb/02a-Memory_Layout_And_The_Stack.php)

